

# Privacy-Preserving Prompted Policy Search for Robotic Control

Ali Irshayyid, Feng Lin, Chong Li and Jun Chen, *Senior Member, IEEE*

**Abstract**—Large language models (LLMs) have recently demonstrated promising capabilities as in-context policy optimizers for Reinforcement Learning (RL), enabling policy search driven by both numerical reward signals and natural language reasoning. However, deploying such methods in practice requires transmitting raw policy parameters and rewards history to cloud-based LLM APIs, exposing proprietary control strategies to third-party service providers. To address this issue, this paper introduces Privacy-Preserving Prompted Policy Search (PP-ProPS), a framework that enables LLM-guided policy optimization while keeping policy and environmental parameters confidential. PP-ProPS encodes policy parameters and reward values using secret client-side transformations before they are included in each API request, ensuring that the LLM provider observes only encoded policy parameters and scaled reward information. Furthermore, unlike Vanilla ProPS, the proposed framework does not require the true optimal episodic return to be known or disclosed to the LLM. Beyond protecting the optimization data, PP-ProPS improves the search process in two ways. First, it provides the LLM with individual reward components instead of only a single total return, offering more informative feedback about each candidate policy. Second, it uses a bounded history that prevents the prompt from growing indefinitely, improving search with high-dimensional policies and supporting the use of open-weight LLMs. The proposed PP-ProPS is evaluated on both continuous and discrete control problems spanning Multi-Joint dynamics with Contact (MuJoCo) locomotion, classic control, highway driving, and robotic arm manipulation. Compared to Vanilla ProPS, the proposed PP-ProPS outperforms ProPS in seven of the ten evaluated tasks, and surpasses conventional RL methods including PPO, SAC, and TRPO, in five of the six tasks.

**Index Terms**—Large language models, reinforcement learning, machine learning for robot control, privacy-preserving optimization, motion control.

## I. INTRODUCTION

Reinforcement learning (RL) enables autonomous agents to learn control policies through interactions with an environment and has achieved notable results in strategic games [1], robotics [2], battery pack control [3], and autonomous vehicles [4]. Traditional RL methods, however, primarily learn from numerical feedback expressed as scalar rewards [5]. In contrast, humans often combine numerical outcomes with language, prior knowledge, and common sense when learning to perform new tasks [6]. Real-world control tasks may also be accompanied by domain descriptions, expert instructions, and operational constraints that conventional RL methods

cannot directly exploit. Prior robotic-learning studies have investigated natural-language instructions for initializing RL policies and language-conditioned policy learning for long-horizon manipulation tasks [7], [8]. Such information can provide useful inductive biases by guiding exploration and providing task-specific constraints.

Recent studies have shown that large language models (LLMs) possess capabilities that extend beyond language generation [9]. In addition to understanding natural language instructions, LLMs can recognize patterns from in-context demonstrations and reason over previously evaluated solutions [10]. This capability enables in-context numerical optimization, in which the problem is described through a prompt and the LLM iteratively generates candidate solutions based on previously observed solution–score pairs. Unlike gradient-based optimization, this process does not require the objective function or its derivatives to be available to the LLM. Instead, the LLM iteratively uses the performance of previously evaluated candidates to propose solutions with higher objective values.

Building on this capability, Prompted Policy Search (ProPS) [11] places an LLM directly within the RL policy-optimization loop. Rather than using the LLM only to design rewards [12], generate high-level robot task plans [13], or generate control actions [14], ProPS uses the LLM to generate and refine the policy parameters directly. Related work has also used LLMs to generate numerical motion parameters for expressive robot behaviors [15]. However, this approach does not iteratively optimize a control policy using feedback obtained from environment interactions. In Vanilla ProPS, at each iteration, the LLM receives a history of previously evaluated parameters and their associated episodic rewards, and then proposes a new parameter vector intended to improve policy performance. The proposed policy is executed independently in the environment, and the resulting reward is returned to the LLM as feedback for the next iteration. Vanilla ProPS additionally assumes that the true optimal episodic return is known in advance and includes this value in the prompt as a target for the optimization. This assumption may be restrictive in practical control problems, where the optimal achievable return is generally unknown and may vary with the system configuration or operating conditions. This formulation treats policy optimization as an in-context reasoning problem and allows numerical reward information to be combined with task descriptions, domain knowledge, and human-provided guidance.

Despite these advantages, the deployment model underlying ProPS introduces a practical concern that requires attention. When a cloud-hosted LLM is used as the optimizer, the raw policy parameters and their corresponding rewards must be repeatedly included in Application Programming Interface

This work is supported in part by National Science Foundation through Award #2432098 and #2432099. *J. Chen is the corresponding author.*

Ali Irshayyid and Jun Chen are with the Department of Electrical and Computer Engineering, Oakland University, Rochester, MI 48309 USA (e-mail: {aliirshayyid,junchen}@oakland.edu). Feng Lin is with the Department of Electrical and Computer Engineering, Wayne State University, Detroit, MI 48202, USA (e-mail: flin@wayne.edu). Chong Li is with OORT and the Department of Electrical Engineering, Columbia University, NYC, 10027, USA (email: cl3607@columbia.edu).

(API) requests. In robotics, autonomous driving, and industrial control, these parameters may represent proprietary control strategies or encode information about the behavior and design of a system [16]. Moreover, the sequence of parameter–reward evaluations may reveal not only individual candidate policies but also how the policy evolves and which parameter configurations produce desirable behavior. Existing LLM-based policy-search methods optimize policies using scalar [11], [17]–[19] or trajectory-level feedback [20] but do not consider how the optimization process can be performed when the underlying policy must remain confidential.

To address this limitation, we introduce Privacy-Preserving Prompted Policy Search (PP-ProPS), an LLM-guided policy-optimization framework that operates in an encoded parameter space. Before each LLM query, the trusted client applies a secret coordinate-wise transformation to the policy parameters and scales the reward-valued feedback using a secret transformation. The LLM therefore receives only encoded policy parameters and scaled reward information, while parameter decoding, policy execution, and environment interaction remain on the client. PP-ProPS also removes the need to provide the true optimal return and performs the search using only the feedback from previously evaluated candidates. PP-ProPS further provides component-level reward and diagnostic feedback to describe why a candidate policy succeeds or fails. A bounded-history representation is also introduced to limit prompt growth and support policy search using open-weight LLMs. The contributions of the paper are summarized as follows.

- Unlike existing LLM-based policy-search methods that expose the original policy or control parameters [11], [19], [20], we introduce PP-ProPS, which applies secret client-side transformations to the policy parameters and reward-valued feedback. The external LLM observes only encoded parameters and scaled rewards, while the original values and encoding variables remain local.
- In contrast to prior LLM-based optimization methods that operate in the original parameter space [11], [17], we demonstrate effective policy search in an encoded parameter-space for policies containing up to 782 parameters. PP-ProPS outperforms Vanilla ProPS in seven of the ten evaluated environments while remaining comparable in the rest.
- Compared with the scalar episodic feedback used by ProPS [11], we provide component-level reward and diagnostic feedback that improves the average policy-search performance in nine of the ten evaluated environments.
- Unlike existing literature methods that retain the full optimization history in each prompt [11], [19], [20], we use a bounded top- $K$  history representation that prevents prompt growth with the number of iterations. Retaining the ten highest-reward candidates reduces the final prompt size and LLM response time by 80.2% and 74.9%, respectively, while maintaining comparable policy-search performance and supporting policy search with a 20B-

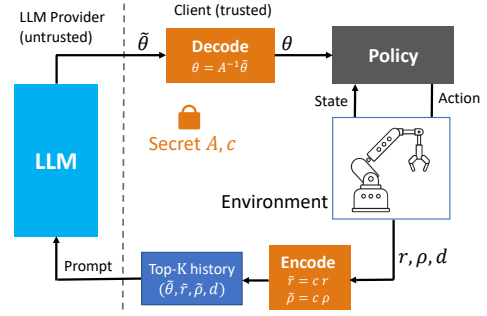


Fig. 1. Overview of the proposed PP-ProPS framework. The LLM searches in the encoded parameter space  $\hat{\theta}$ , while decoding, policy execution, reward scaling, and storage of the secret variables  $A$  and  $c$  are performed on the trusted client.

parameter open-weight LLM.

## II. METHODOLOGY

This section first formulates Vanilla ProPS and then presents the parameter- and reward-encoding transformations, component-level reward feedback, bounded-history representation, and LLM-guided policy-search procedure.

### A. Problem formulation

We consider episodic RL with a parameterized policy  $\pi_{\theta}$ , where  $\theta \in \mathbb{R}^D$  denotes the policy parameters. For a trajectory  $\tau$  generated by  $\pi_{\theta}$  (denoted by  $\tau \sim \pi_{\theta}$ ), let  $R(\tau)$  denote its cumulative episodic return. The policy objective is

$$\theta^* \in \arg \max_{\theta} J(\theta), \quad J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} [R(\tau)]. \quad (1)$$

ProPS [11] uses an LLM,  $\mathcal{M}$ , as the policy optimizer. Let  $\mathcal{P}$  denote the fixed prompt information, including the task description, policy structure, parameter constraints, optimal value of  $J(\theta)$ , and output-format instructions. After  $i$  candidate policies have been evaluated, the optimization history containing the parameter-return pair from each iteration  $j = 1, \dots, i$  is defined as

$$\Gamma^{(i)} = \{(\theta_j, r_j)\}_{j=1}^i, \quad r_j = R(\tau_j), \quad \tau_j \sim \pi_{\theta_j}. \quad (2)$$

At the next optimization iteration, the LLM generates a new candidate according to

$$\theta_{i+1} = \mathcal{M}(\mathcal{P}, \Gamma^{(i)}), \quad (3)$$

where  $\mathcal{M}(\cdot)$  denotes the parameter vector extracted from the LLM response. The candidate policy is then executed in the environment, and its episodic return is appended to the history  $\Gamma^{(i+1)} = \Gamma^{(i)} \cup \{(\theta_{i+1}, r_{i+1})\}$ . Thus, ProPS performs policy search through repeated LLM queries without requiring policy gradients, a value function, or an environment model within the optimization procedure.

To preserve privacy, the proposed PP-ProPS instead transmits only an encoded optimization history to the LLM. The original parameters  $\theta$ , secret encoding variables  $A$  and  $c$ , policy execution, and all environment interactions remain on the trusted client, as illustrated in Fig. 1.

## B. Parameter Encoding

At the beginning of each optimization run, the client generates a secret positive diagonal encoding matrix

$$A = \text{diag}(a_1, a_2, \dots, a_D), \quad a_j > 0, \quad (4)$$

where each scaling coefficient  $a_j$  is independently sampled. A new matrix is generated for every independent optimization run and remains fixed across all LLM queries within that run. Maintaining the same matrix provides a consistent encoded coordinate system from which the LLM can identify relationships between parameter changes and policy performance. Before transmission, the original policy vector is encoded as

$$\tilde{\theta} = A\theta, \quad (5)$$

where  $\tilde{\theta} \in \mathbb{R}^D$  denotes the encoded parameter vector observed by the LLM. Because  $a_j > 0$  for all  $j$ , the matrix  $A$  is invertible. After the LLM generates an encoded candidate  $\tilde{\theta}_i$ , the client recovers the corresponding policy parameters locally using

$$\theta_i = A^{-1}\tilde{\theta}_i. \quad (6)$$

The diagonal transformation preserves the policy layout and parameter identities while introducing an unknown scaling factor for each parameter dimension. Consequently, the LLM can continue to use the task description, policy structure, and relationships observed in the optimization history, but it can never infer the true policy parameters deployed for robotic control.

## C. Component-Level Feedback and Reward Encoding

The total episodic return provides a scalar measure of candidate policy performance but may not indicate which aspects of the behavior contribute to the observed result. Many control environments define the reward using interpretable terms associated with objectives such as forward progress, control effort, healthy behavior, collision avoidance, and task success. Providing these quantities separately allows the LLM to distinguish between policies that achieve similar total returns through different behaviors.

For each evaluated candidate, PP-ProPS therefore reports the scalar episodic return together with the available component-level reward information. Let  $\rho_i = [\rho_{i,1}, \rho_{i,2}, \dots, \rho_{i,Q}]$  denote the additional feedback associated with candidate  $i$ , where  $Q$  is the number of reward components and  $\rho_{i,q}$  represents the episodic contribution of the  $q$ th reward term. Task-specific diagnostic quantities vector  $d_i$ , such as episode length or success indicators, is reported separately. The exact feedback provided for each environment is summarized in Table I.

Although parameter encoding prevents the direct transmission of the original policy parameters, the absolute reward magnitude may provide information regarding the operating condition or the performance of the evaluated policy relative to a known benchmark. To obscure the direct reward magnitude, the client samples a secret positive scaling factor  $c$  at the beginning of each optimization run. The same factor remains

fixed throughout the run and is applied to the reported reward values:

$$\tilde{r}_i = cr_i, \quad c > 0. \quad (7)$$

The same scaling factor is applied to each reward-valued component:

$$\tilde{\rho}_{i,q} = c\rho_{i,q}, \quad q = 1, \dots, Q. \quad (8)$$

Because  $c$  is positive, reward scaling preserves the ranking of the evaluated candidates and therefore does not change the policy selected as the best performing candidate. In other words, reward encoding obscures the direct absolute reward scale while retaining the information required to compare candidate policies.

*Remark 1:* Vanilla ProPS [11] assumes that the true optimal return is known in advance and provides this value in the prompt to guide the LLM. In the complete PP-ProPS configuration, the LLM receives no information about the true optimal return.

## D. Bounded History Representation

Existing prompt-based policy-search methods append each evaluated candidate to the optimization history  $\Gamma^{(i)}$  included in the prompt [11], [19]. Because each history entry contains a complete policy vector and its associated reward feedback, the prompt size grows with both the number of evaluated candidates and the policy dimension  $D$ , which may become prohibitive for long optimization runs or high-dimensional policies.

To limit this growth, the proposed PP-ProPS maintains the complete optimization encoded history locally but includes only the  $K$  highest-reward candidates in each LLM prompt. Let  $\tilde{\Gamma}^{(i)} = \{(\tilde{\theta}_j, \tilde{r}_j, \tilde{\rho}_j, d_j)\}_{j=1}^i$  denote the complete encoded history after  $i$  candidate evaluations. The bounded history is obtained as

$$\tilde{\Gamma}_K^{(i)} = \text{Sort}_{\uparrow} \left( \text{TopK}_{\tilde{r}} \left( \tilde{\Gamma}^{(i)}, \min \left( K, \left| \tilde{\Gamma}^{(i)} \right| \right) \right) \right), \quad (9)$$

where  $\text{TopK}(\cdot, k)$  returns the  $k$  history entries with the highest encoded rewards, and  $\text{Sort}_{\uparrow}(\cdot)$  orders the selected entries from the lowest to the highest encoded reward such that the best-performing candidate appears last in the prompt. The minimum operator handles early iterations in which fewer than  $K$  candidates have been evaluated. Because the reward scaling factor  $c$  is positive, ranking candidates by  $\tilde{r}$  produces the same ordering as ranking them by the original rewards  $r$ . If each history entry contains  $D$  encoded parameters, the history requirement of the bounded representation is  $\mathcal{O}(KD)$ . In comparison, an unbounded history containing  $H_i$  evaluated candidates requires  $\mathcal{O}(H_i D)$ , where  $H_i$  is the number of candidates evaluated by iteration  $i$ . Therefore, the bounded representation maintains a fixed history size in the prompt as the number of optimization iterations increases.

### PP-ProPS Prompt

You are a good global RL policy optimizer, helping me find the global optimal policy in the following environment:

1. The task description: % Environment, state and action semantics, policy structure, and hints
2. The policy parameters: % parameters are encoded
3. The feedback: % scaled return and component-level terms
4. How we will interact: % formatting and output instructions
5. Here is the history: % bounded top- $K$ , best entry last

Fig. 2. Summary of the PP-ProPS prompt structure.

### E. Privacy-Preserving Prompted Policy Search

Algorithm 1 summarizes the complete PP-ProPS combining prompt policy search, parameters and rewards encoding, component-level reward feedback, and bounded history representation. At the beginning of each optimization run, the client generates the secret parameter-encoding matrix  $A$  and reward scaling factor  $c$ , which remain fixed throughout the run. Let  $N_w$  and  $N_o$  denote the numbers of randomly sampled warm-up policies and LLM-guided optimization iterations, respectively, and let  $\mathcal{E}$  denote the execution environment. The first  $N_w$  policies are sampled from the feasible parameter range and evaluated to initialize the encoded history  $\tilde{\Gamma}$ . Each history entry contains the encoded policy parameters, scaled scalar and component-level rewards, and any non-reward diagnostic feedback. The original returns remain local and are used only to identify the best performing policy.

At iteration  $i$ , the client retains  $K$  candidates with the highest encoded returns and orders them from lowest to highest return. This history is combined with the task description, policy structure, and behavioral guidance to form the prompt  $\mathcal{P}_i$  (illustrated in Fig. 2). The LLM generates a candidate  $\tilde{\theta}_i$  in the encoded space, which the client decodes using (6), evaluates in the environment, and appends the resulting encoded parameter, reward, component-feedback, and diagnostic tuple to  $\tilde{\Gamma}$ . Unlike ProPS [11], the prompt does not disclose the true optimal achievable return. Note that the privacy-preserving transformation is only needed during optimization. Once the (decoded) best policy  $\pi_{\theta_{best}}$  is found, it can be deployed for robot control directly without requiring any encoding during execution.

### III. EXPERIMENTAL SETUP

The experiments evaluate four aspects of PP-ProPS: the effect of parameter encoding, the contribution of component-level feedback, the performance of the complete parameter- and reward-encoded formulation, and the tradeoff between bounded-history performance and prompt size. The LLM-guided configurations are also compared with standard RL algorithms.

We consider ten environment configurations spanning four domains, as shown in Fig. 3 and summarized in Table I, MuJoCo locomotion [21] (Ant-v5 [22], Humanoid-v5 [23]), classic control tasks (Acrobot-v1 [24], CartPole-v1 [25]),

### Algorithm 1: Privacy-Preserving Prompted Policy Search (PP-ProPS)

---

```

1 Initialize  $A$ ,  $a_j > 0$ ,  $c > 0$ ,  $\tilde{\Gamma} \leftarrow \emptyset$ ,  $r_{best} \leftarrow -\infty$ , and  $\theta_{best} \leftarrow \emptyset$ ;
2 for  $m = 1$  to  $N_w$  do
3   Sample  $\theta_m$  from the feasible parameter range;
4   Execute  $\pi_{\theta_m}$  in  $\mathcal{E}$  and obtain  $r_m$ ,  $\rho_m$ , and  $d_m$ ;
5    $\tilde{\theta}_m \leftarrow A\theta_m$ ;
6    $\tilde{r}_m \leftarrow cr_m$  and  $\tilde{\rho}_m \leftarrow c\rho_m$ ;
7    $\tilde{\Gamma} \leftarrow \tilde{\Gamma} \cup \{(\tilde{\theta}_m, \tilde{r}_m, \tilde{\rho}_m, d_m)\}$ ;
8   if  $r_m > r_{best}$  then
9      $r_{best} \leftarrow r_m$  and  $\theta_{best} \leftarrow \theta_m$ ;
10  end
11 end
12 for  $i = 1$  to  $N_o$  do
13    $K_i \leftarrow \min(K, |\tilde{\Gamma}|)$ ;
14    $\tilde{\Gamma}_K \leftarrow \text{Sort}^\uparrow(\text{TopK}_{\tilde{\Gamma}}(\tilde{\Gamma}, K_i))$ ;
15   Construct  $\mathcal{P}_i$  using  $\tilde{\Gamma}_K$ ;
16    $\tilde{\theta}_i \leftarrow \mathcal{M}(\mathcal{P}_i)$ ;
17    $\theta_i \leftarrow A^{-1}\tilde{\theta}_i$ ;
18   Execute  $\pi_{\theta_i}$  in  $\mathcal{E}$  and obtain  $r_i$ ,  $\rho_i$ , and  $d_i$ ;
19    $\tilde{r}_i \leftarrow cr_i$  and  $\tilde{\rho}_i \leftarrow c\rho_i$ ;
20    $\tilde{\Gamma} \leftarrow \tilde{\Gamma} \cup \{(\tilde{\theta}_i, \tilde{r}_i, \tilde{\rho}_i, d_i)\}$ ;
21   if  $r_i > r_{best}$  then
22      $r_{best} \leftarrow r_i$  and  $\theta_{best} \leftarrow \theta_i$ ;
23   end
24 end
25 Return  $\theta_{best}$ ;
```

---

robotic arm manipulation (FetchReachDense-v4 [26]), and highway driving (highway-v0 [27]). The environments include both continuous and discrete action spaces. For Ant-v5 and Humanoid-v5, three observation configurations, denoted as Ultra, Joints, and Torso, progressively expand the observation vector while preserving the underlying control task. Ultra contains selected body, joint, and velocity variables; Joints adds locomotion-related joint states; and Torso contains the complete generalized position and velocity states. The corresponding policy dimensions range from 88 for Ant-Ultra to 782 for Humanoid-Torso.

A linear policy is adopted for all evaluated environments, consistent with benchmark Vanilla ProPS [11]. For an environment with  $n_s$  observations and  $n_a$  actions, the policy is defined as

$$\mathbf{u} = W^T \mathbf{s} + \mathbf{b}, \quad (10)$$

where  $\mathbf{s} \in \mathbb{R}^{n_s}$  denotes the observation vector,  $W \in \mathbb{R}^{n_s \times n_a}$  is the policy weight matrix,  $\mathbf{b} \in \mathbb{R}^{n_a}$  is the bias vector, and  $\mathbf{u} \in \mathbb{R}^{n_a}$  is the policy output. For continuous-action environments,  $\mathbf{u}$  is applied as the actuation vector subject to the environment action limits. For discrete-action environments, the elements of  $\mathbf{u}$  are treated as action scores, and the action with the highest

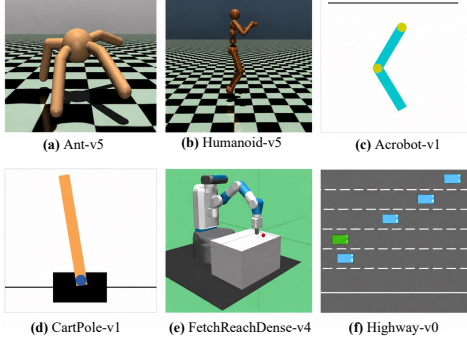


Fig. 3. Environments used to evaluate the proposed PP-ProPS framework.

score is executed. The resulting policy-parameter vector has dimension  $D = (n_s + 1)n_a$ . To isolate the contribution of each mechanism introduced, four configurations are evaluated in this study as follows.

- **Vanilla ProPS** [11]: In this configuration, the LLM observes the true parameters  $\theta$ , and each candidate policy performance is evaluated by its scalar episodic reward. The prompt discloses the reward function structure and the true maximum achievable return.
- **Parameter-Encoded ProPS (PE-ProPS)**: This configuration isolates the effect of parameter encoding. The task description, policy structure, and scalar reward feedback are identical to those of Vanilla ProPS. However, each policy vector is encoded using the secret diagonal transformation  $A$  in (5) before being included in the prompt. The original policy parameters remain local to the client.
- **Detailed-Feedback ProPS (DF-ProPS)**: This configuration evaluates the contribution of additional policy-performance information without applying parameter or reward encoding. The LLM observes the original policy parameters and scalar episodic return, together with the component-level rewards and task-specific feedback summarized in Table I.
- **Privacy-Preserving Prompted Policy Search (PP-ProPS)**: This configuration represents the complete proposed framework. The policy parameters are encoded according to (5), while the scalar episodic return and the reward-valued feedback terms are scaled using the secret factor defined in (7). The detailed feedback used by DF-ProPS is retained, whereas the maximum achievable return is withheld from the prompt.

Reflective Prompted Policy Optimization (R2PO) [20] is also evaluated as a recent baseline in which a Search-LLM proposes initial policy, a Critic-LLM revises it using trajectory evidence and the higher-return candidate is retained. R2PO uses the same 10-policy random warm-up and 25 two-stage iterations to match the 50 LLM calls used by the ProPS-based configurations. Following [20], each initial and revised candidate is evaluated over 20 episodes.

All LLM-guided policy-search configurations use gpt-oss-20b open-weight reasoning model [28], with

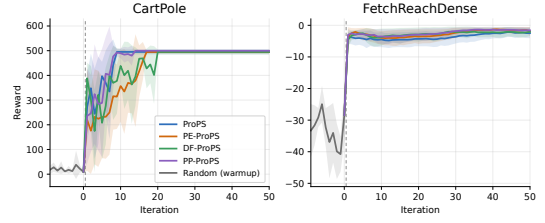


Fig. 4. Optimization performance of the evaluated ProPS configurations across two environments. Curves represent the mean episodic rewards and shaded regions indicate the standard deviation over five independent runs. Iterations preceding  $i = 0$  correspond to the 10-policy random warm-up phase.

identical inference settings across all configurations. For PE-ProPS and PP-ProPS, a new encoding matrix  $A$  is independently generated at the beginning of each run and remains fixed throughout that run. PP-ProPS additionally generates a new reward-scaling factor  $c$  at the beginning of each run. Unless otherwise stated, all ProPS-based configurations use the bounded top- $K$  history with  $K = 10$ . The bounded and unbounded history-formulations are compared in Section IV-A.

In addition, seven standard RL algorithms are evaluated using Stable-Baselines3 [29] default hyperparameters: Proximal Policy Optimization (PPO) [30], Advantage Actor-Critic (A2C) [31], Trust-Region Policy Optimization (TRPO) [32], Soft Actor-Critic (SAC) [33], Twin-Delayed Deep Deterministic Policy Gradient (TD3) [34], Deep Deterministic Policy Gradient (DDPG) [35], and Deep Q-Network (DQN) [36]. SAC, TD3, and DDPG are applied only to environments with continuous action spaces, whereas DQN is applied only to environments with discrete action spaces. Inapplicable combinations are indicated by “–” in Table IV. The Ant-v5 and Humanoid-v5 baseline results reported in Table IV use the Ultra observation configuration.

## IV. RESULTS

This section evaluates the bounded-history representation, the effect of parameter encoding, the contribution of component-level feedback, and performance relative to R2PO and standard RL algorithms. Unless otherwise stated, results are reported in Table II as the mean episodic return of the best policy found under each method’s stated evaluation protocol, with the standard deviation across five independent runs shown in parentheses. All LLM-based methods use a 10-policy random warm-up. The best result for each environment is highlighted in bold.

Fig. 4 illustrates the reward curves for two representative environments. Following the random warm-up phase, all four configurations rapidly approach the maximum CartPole return of 500, whereas improvement is more gradual for FetchReachDense. PP-ProPS follows trends comparable to the unencoded configurations despite receiving only encoded policy parameters and rewards.



TABLE I  
EVALUATED ENVIRONMENTS, POLICY DIMENSIONS, AND DETAILED FEEDBACK PROVIDED TO THE LLM

| Environment          | Action type | $n_a$ | $n_s$ | Detailed feedback                                                                        | $D$ |
|----------------------|-------------|-------|-------|------------------------------------------------------------------------------------------|-----|
| Ant-v5 (Ultra)       | Continuous  | 8     | 10    | Forward-progress reward, healthy reward, control-cost penalty, and contact-force penalty | 88  |
| Ant-v5 (Joints)      | Continuous  | 8     | 16    | Same as Ant-v5 (Ultra)                                                                   | 136 |
| Ant-v5 (Torso)       | Continuous  | 8     | 27    | Same as Ant-v5 (Ultra)                                                                   | 224 |
| Humanoid-v5 (Ultra)  | Continuous  | 17    | 11    | Forward-progress reward, healthy reward, control-cost penalty, and contact-force penalty | 204 |
| Humanoid-v5 (Joints) | Continuous  | 17    | 26    | Same as Humanoid-v5 (Ultra)                                                              | 459 |
| Humanoid-v5 (Torso)  | Continuous  | 17    | 45    | Same as Humanoid-v5 (Ultra)                                                              | 782 |
| Acrobot-v1           | Discrete    | 3     | 6     | Episode length, task-success indicator, and average height of the free-tip               | 21  |
| CartPole-v1          | Discrete    | 2     | 4     | Episode length, and termination cause                                                    | 10  |
| FetchReachDense-v4   | Continuous  | 4     | 6     | Episode length and task-success indicator                                                | 28  |
| Highway-v0           | Discrete    | 5     | 15    | High-speed reward, right-lane reward, collision penalty, and on-road reward              | 80  |

TABLE II  
COMPARISON OF EPISODIC REWARDS FOR VANILLA PROPS, R2PO, PE-PROPS, DF-PROPS, AND PP-PROPS ACROSS ALL EVALUATED ENVIRONMENTS

| Environment     | Vanilla ProPS         | R2PO                        | PE-ProPS       | DF-ProPS              | PP-ProPS              |
|-----------------|-----------------------|-----------------------------|----------------|-----------------------|-----------------------|
| Ant-Ultra       | 942.75 (59.11)        | Context overflow            | 998.64 (4.75)  | <b>998.9</b> (37.5)   | 994.55 (10.72)        |
| Ant-Joints      | 966.58 (48.61)        | Context overflow            | 982.62 (26.45) | <b>996.15</b> (2.79)  | 989.68 (17.11)        |
| Ant-Torso       | 929.19 (61.73)        | Context overflow            | 982.41 (28.62) | 993.76 (0.27)         | <b>997.08</b> (7.69)  |
| Humanoid-Ultra  | 448.2 (51.65)         | 443.7 (80.5)                | 382.51 (73.79) | <b>475.91</b> (72.61) | 375.52 (96.64)        |
| Humanoid-Joints | <b>394.38</b> (90.88) | 293.2 (77.1)                | 364.5 (94.03)  | 387 (69.3)            | 361.59 (97.2)         |
| Humanoid-Torso  | 307.38 (63.7)         | 260.92 (20.74) <sup>†</sup> | 251 (54.93)    | <b>326.43</b> (87.67) | 290.68 (43.57)        |
| Acrobot         | -77.6 (1.39)          | -83.5 (4.6)                 | -77.4 (0.2)    | <b>-71.6</b> (0.4)    | -73.3 (1.12)          |
| CartPole        | 495 (10)              | <b>500</b> (0.0)            | 494.8 (6.5)    | 495.2 (10)            | 499.9 (0.2)           |
| FetchReachDense | -0.6 (0.03)           | -0.61 (0.07)                | -0.78 (0.06)   | <b>-0.51</b> (0.05)   | -0.54 (0.03)          |
| Highway Env     | 122.57 (24.25)        | 146.2 (1.4)                 | 90.04 (18.86)  | 151.46 (4.16)         | <b>154.55</b> (10.33) |

<sup>†</sup>Three runs terminated early because of context overflow. The best pre-overflow policies are reported.

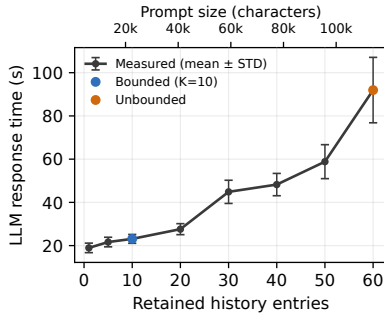


Fig. 5. LLM response time for Ant-Ultra ( $D = 88$ ) versus retained history size and the corresponding prompt size. Highlighted points denote bounded ( $K = 10$ ) and unbounded history representations.

#### A. Bounded-History Analysis

Using a 20B-parameter open-weight LLM makes prompt size a practical concern, particularly for policies containing up to 782 parameters, more than seven times the dimension evaluated in Vanilla ProPS [11]. Table III compares the bounded and unbounded Vanilla ProPS history approaches for the three Ant policies. The unbounded history improves the return by only 0.84%, 0.03%, and 0.38% for  $D = 88$ , 136, and 224, respectively. Thus, the top- $K$  representation maintains performance comparable to the complete history.

Such a slight performance degradation, however, is re-

TABLE III  
COMPARISON OF THE BOUNDED AND UNBOUNDED HISTORY REPRESENTATIONS ACROSS THE ANT POLICY DIMENSIONS

| $D$ | Bounded history ( $K = 10$ ) | Unbounded history    | Diff (%) |
|-----|------------------------------|----------------------|----------|
| 88  | 942.75 (59.11)               | <b>950.75</b> (32.1) | -0.84    |
| 136 | 966.58 (48.61)               | <b>966.9</b> (15.7)  | -0.03    |
| 224 | 929.19 (61.73)               | <b>932.7</b> (53.7)  | -0.38    |

warded by significant computation gain. As shown in Fig. 5, the top-10 history reduces the final prompt size from 114,000 to 22,000 characters and the mean LLM response time by 74.9%, corresponding to an 80.2% prompt size reduction. Accordingly, the bounded top- $K$  representation with  $K = 10$  is used for all other LLM-guided evaluations, unless otherwise stated.

#### B. Effect of Parameter Encoding

The Vanilla ProPS and PE-ProPS columns of Table II isolate the effect of parameter encoding. Encoding improves all three Ant configurations, increasing their average returns by approximately 1.7-5.9% while generally reducing run-to-run variability. For Ant-Ultra, the return increases from 942.75 to 998.64 while the standard deviation decreases from 59.11 to 4.75. These results show that the LLM can identify useful parameter-reward relationships without observing the original parameter magnitudes. We hypothesize that encoding

reduces the influence of pretrained assumptions about parameter magnitudes. Under Vanilla ProPS, the LLM may associate particular policy-parameter values with expected control behavior based on its prior knowledge of the environment or related dynamical systems. Such prior associations may bias candidate generation when the relationship between parameter magnitude and episodic return differs from those expectations. The encoded representation encourages the LLM to rely primarily on the numerical relationships observed during the optimization process.

For Humanoid, encoding reduces the average return across all three observation configurations. Nevertheless, PE-ProPS continues to generate effective policies for dimensions up to 782, indicating that encoded in-context policy search remains feasible at the evaluated scale. CartPole and Acrobot are nearly unchanged, whereas the average returns for FetchReachDense and Highway environments decrease from  $-0.60$  to  $-0.78$  and  $122.57$  to  $90.04$ , respectively. These results suggest that, after parameter magnitudes are obscured, scalar feedback may be insufficient for tasks requiring more informative behavioral guidance, motivating the component-level feedback evaluated next.

### C. Effect of Component-Level Reward Feedback

The Vanilla ProPS and DF-ProPS columns of Table II show that component-level and diagnostic feedback improves the average return in nine of the ten environments. For the three Ant configurations, the improvements range from 3.1% to 6.9%, with the standard deviations decreasing to 2.79 and 0.27 for Ant-Joints and Ant-Torso, respectively. DF-ProPS also improves Humanoid-Ultra and Humanoid-Torso by approximately 6.2%, while the small decrease for Humanoid-Joints remains limited relative to the reported variation. The feedback, summarized in Table I, provides information regarding forward motion, control effort, health, and contact penalties, allowing the LLM to distinguish between policies that obtain similar total returns through different behaviors.

The benefit of detailed feedback is also evident in the remaining control environments. For Acrobot, the average return improves from  $-77.60$  to  $-71.60$ . For CartPole, the return increases from 495 to 495.2. The FetchReachDense return improves from  $-0.60$  to  $-0.51$ . A substantial improvement is observed for Highway, where component-level feedback increases the average return from 122.57 to 151.46, corresponding to an improvement of approximately 23.6%.

### D. Performance of Complete PP-ProPS

The final column of Table II evaluates the complete PP-ProPS configuration, in which the policy parameters and reward-valued feedback are encoded, component-level feedback is provided, and the optimal achievable return is withheld from the prompt (unlike Vanilla ProPS). Despite these additional privacy mechanisms, PP-ProPS outperforms Vanilla ProPS in seven of the ten evaluated environments. The largest gains occur for Ant-Torso and Highway, which improve by approximately 7.3% and 26.1%, respectively. PP-ProPS also

improves Acrobot and FetchReachDense and achieves a Cart-Pole return of 499.9 with the lowest variability among the four configurations.

Performance decreases for the three Humanoid configurations, suggesting that the effectiveness of the complete encoded formulation also depends on the task dynamics and informativeness of the available feedback. Among the seven tasks with reported R2PO results, PP-ProPS outperforms R2PO in five. For Ant, the long episode horizon causes the trajectory prompt to exceed the context window before the first Critic-LLM query. Additionally, three of the five Humanoid-Torso runs also terminate before completing all iterations, and their reported returns use the best policy found before the overflow. In contrast, PP-ProPS completes all evaluations without exposing the original parameters or reward scale.

### E. Standard RL Baselines

Table IV compares the proposed PP-ProPS with seven standard RL algorithms. For Ant-Ultra, PPO achieves the highest return of 1000.16, while DF-ProPS, PE-ProPS, and PP-ProPS remain within approximately 0.6% of this result. Furthermore, PE-ProPS achieves a standard deviation of 4.75, compared with 15.06 for PPO. Therefore, the proposed PP-ProPS configurations maintain comparable performance while directly optimizing a compact linear policy.

The proposed PP-ProPS approaches outperform all evaluated RL baselines for Humanoid-Ultra, Acrobot, CartPole, Highway, and FetchReachDense. DF-ProPS more than doubles the best RL-baseline return for Humanoid-Ultra, while PP-ProPS improves the Highway return by approximately 41.7% relative to TRPO. For FetchReachDense, DF-ProPS achieves  $-0.51$ , compared with  $-7.39$  for DDPG, the strongest evaluated RL baseline. Thus, the LLM-guided configurations remain competitive on Ant-Ultra and provide substantially higher returns across all remaining continuous- and discrete-control tasks. Furthermore, PP-ProPS preserves strong policy-search performance while maintaining the privacy-preserving parameter and reward representations.

## V. CONCLUSION

This paper presents Privacy-Preserving Prompted Policy Search (PP-ProPS), an LLM-guided policy-optimization framework that avoids transmitting raw policy parameters and reward information to an external LLM provider. The proposed framework combines client-side parameter and reward transformations with component-level feedback and a bounded-history representation. Evaluation across ten continuous- and discrete-control configurations demonstrates that PP-ProPS outperforms Vanilla ProPS in seven environments and remains effective for policies containing up to 782 parameters, while reducing the final prompt size and mean LLM response time by approximately 80.2% and 74.9%, respectively. Furthermore, the proposed framework outperforms the evaluated reinforcement learning baselines in five of the six task groups and degrades by only 0.6% for the remaining one. Future work will evaluate nonlinear control policies with physical experiments

TABLE IV  
PERFORMANCE COMPARISON OF THE PROPOSED PP-PROPS AND STANDARD RL BASELINES

| Approach | Ant                    | Humanoid              | Acrobot            | CartPole           | Highway               | FetchReachDense     |
|----------|------------------------|-----------------------|--------------------|--------------------|-----------------------|---------------------|
| PPO      | <b>1000.16</b> (15.06) | 192.94 (16.38)        | -303.95 (170.33)   | 127.16 (179.77)    | 107.01 (26.51)        | -7.65 (0.46)        |
| A2C      | 233.61 (126.42)        | 118.82 (75.77)        | -209.57 (158.84)   | 161.69 (177.61)    | 73.68(33.65)          | -12.80 (8.19)       |
| SAC      | -52.42 (46.51)         | 78.24 (3.10)          | —                  | —                  | —                     | -8.55 (0.99)        |
| TD3      | 938.04 (240.49)        | 81.86 (34.78)         | —                  | —                  | —                     | -8.06 (1.79)        |
| DDPG     | -6.55 (72.99)          | 89.59 (40.25)         | —                  | —                  | —                     | -7.39 (2.45)        |
| TRPO     | 965.23 (10.99)         | 193.21 (16.80)        | -284.65 (171.60)   | 127.04 (187.54)    | 109.09 (30.69)        | -8.76 (0.93)        |
| DQN      | —                      | —                     | -408.80 (150.13)   | 12.51 (5.98)       | 39.14 (4.31)          | —                   |
| PE-ProPS | 998.64 (4.75)          | 382.51 (73.79)        | -77.4 (0.2)        | 494.8 (6.5)        | 90.04 (18.86)         | -0.78 (0.06)        |
| DF-ProPS | 998.90 (37.50)         | <b>475.91</b> (72.61) | <b>-71.6</b> (0.4) | 495.2 (10)         | 151.46 (4.16)         | <b>-0.51</b> (0.05) |
| PP-ProPS | 994.55 (10.72)         | 375.52 (96.64)        | -73.3 (1.12)       | <b>499.9</b> (0.2) | <b>154.55</b> (10.33) | -0.54 (0.03)        |

and investigate nonlinear encoding against parameter-recovery and information-inference attacks.

## REFERENCES

- [1] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. Van Den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot *et al.*, “Mastering the game of go with deep neural networks and tree search,” *nature*, vol. 529, no. 7587, pp. 484–489, 2016.
- [2] B. Wang, Z. Liu, Q. Li, and A. Prorok, “Mobile robot path planning in dynamic environments through globally guided reinforcement learning,” *IEEE Robot. Autom. Lett.*, vol. 5, no. 4, pp. 6932–6939, 2020.
- [3] A. Irshayid, W. Yang, and J. Chen, “Real-time balancing control of reconfigurable battery packs using reinforcement learning,” *IEEE Trans. Transp. Electrification*, vol. 12, no. 4, pp. 7013–7024, August 2026.
- [4] A. Irshayid, J. Chen, and G. Xiong, “A review on reinforcement learning-based highway autonomous vehicle control,” *Green Energy and Intelligent Transportation*, vol. 3, no. 4, p. 100156, 2024.
- [5] R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction*. MIT press, 2018.
- [6] G. Lupyan and B. Bergen, “How language programs the mind,” *Topics in Cognitive Science*, vol. 8, no. 2, pp. 408–424, 2016.
- [7] P. Tambwekar, A. Silva, N. Gopalan, and M. Gombolay, “Natural language specification of reinforcement learning policies through differentiable decision trees,” *IEEE Robot. Autom. Lett.*, vol. 8, no. 6, pp. 3621–3628, 2023.
- [8] O. Mees, L. Hermann, E. Rosete-Beas, and W. Burgard, “Calvin: A benchmark for language-conditioned policy learning for long-horizon robot manipulation tasks,” *IEEE Robot. Autom. Lett.*, vol. 7, no. 3, pp. 7327–7334, 2022.
- [9] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, “Large language models are zero-shot reasoners,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 22 199–22 213, 2022.
- [10] S. Mirchandani, F. Xia, P. Florence, B. Ichter, D. Driess, M. G. Arenas, K. Rao, D. Sadigh, and A. Zeng, “Large language models as general pattern machines,” *arXiv preprint arXiv:2307.04721*, 2023.
- [11] Y. Zhou, S. Grover, M. E. Mistiri, K. Kalirathnam, P. Kerhalkar, S. Mishra, N. Kumar, S. Gaurav, O. Aran, and H. B. Amor, “Prompted policy search: Reinforcement learning through linguistic and numerical reasoning in llms,” *Advances in Neural Information Processing Systems*, 2025.
- [12] N. Turcato, M. Iovino, A. Synodinos, A. Dalla Libera, R. Carli, and P. Falco, “Towards autonomous reinforcement learning for real-world robotic manipulation with large language models,” *IEEE Robot. Autom. Lett.*, 2025.
- [13] Y. Tsushima, S. Yamamoto, A. A. Ravankar, J. V. S. Lucas, and Y. Hirata, “Task planning for a factory robot using large language model,” *IEEE Robot. Autom. Lett.*, vol. 10, no. 3, pp. 2383–2390, 2025.
- [14] Y. Han, M. Yang, Y. Ren, and W. Li, “Large language model guided reinforcement learning based six-degree-of-freedom flight control,” *IEEE Access*, vol. 12, pp. 89 479–89 492, 2024.
- [15] L. Roy, E. A. Croft, A. Ramirez, and D. Kulić, “GPT-driven gestures: Leveraging large language models to generate expressive robot motion for enhanced human-robot interaction,” *IEEE Robot. Autom. Lett.*, 2025.
- [16] S. Gummadi, M. V. Gasparino, D. Vasisht, and G. Chowdhary, “Fed-ec: Bandwidth-efficient clustering-based federated learning for autonomous visual robot navigation,” *IEEE Robot. Autom. Lett.*, vol. 9, no. 12, pp. 11 841–11 848, 2024.
- [17] C. Yang, X. Wang, Y. Lu, H. Liu, Q. V. Le, D. Zhou, and X. Chen, “Large language models as optimizers,” in *International Conference on Learning Representations*, vol. 2024, 2024, pp. 12 028–12 068.
- [18] T. Zhang, J. Yuan, and S. Avestimehr, “Revisiting opro: The limitations of small-scale llms as optimizers,” in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024, pp. 1727–1735.
- [19] H. B. Amor, L. Graesser, A. Iscen, D. D’Ambrosio, S. Abevruwan, A. Bewley, Y. Zhou, K. Kalirathnam, S. Mishra, and P. Sanketi, “Sas-prompt: Large language models as numerical optimizers for robot self-improvement,” in *2025 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025, pp. 10 087–10 094.
- [20] R. Abu Hara, V. Murarri, and C. Zito, “Reflective prompted policy optimization: Trajectory-grounded revision and salience bias,” *arXiv preprint arXiv:2605.08315*, 2026.
- [21] E. Todorov, T. Erez, and Y. Tassa, “MuJoCo: A physics engine for model-based control,” in *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, 2012, pp. 5026–5033.
- [22] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, “High-dimensional continuous control using generalized advantage estimation,” *arXiv preprint arXiv:1506.02438*, 2015.
- [23] Y. Tassa, T. Erez, and E. Todorov, “Synthesis and stabilization of complex behaviors through online trajectory optimization,” in *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, 2012, pp. 4906–4913.
- [24] R. S. Sutton, “Generalization in reinforcement learning: Successful examples using sparse coarse coding,” *Advances in Neural Information Processing Systems*, vol. 8, 1995.
- [25] A. G. Barto, R. S. Sutton, and C. W. Anderson, “Neuronlike adaptive elements that can solve difficult learning control problems,” *IEEE Trans. Syst. Man Cybern.*, no. 5, pp. 834–846, 1983.
- [26] M. Plappert, M. Andrychowicz, A. Ray, B. McGrew, B. Baker, G. Powell, J. Schneider, J. Tobin, M. Chociej, P. Welinder, V. Kumar, and W. Zaremba, “Multi-goal reinforcement learning: Challenging robotics environments and request for research,” *arXiv preprint arXiv:1802.09464*, 2018.
- [27] E. Leurent, “An Environment for Autonomous Driving Decision-Making,” May 2018. [Online]. Available: <https://github.com/Farama-Foundation/HighwayEnv>
- [28] S. Agarwal, L. Ahmad, J. Ai, S. Altman, A. Applebaum, E. Arbus, R. K. Arora, Y. Bai, B. Baker, H. Bao *et al.*, “gpt-oss-120b & gpt-oss-20b model card,” *arXiv preprint arXiv:2508.10925*, 2025.
- [29] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, “Stable-baselines3: Reliable reinforcement learning implementations,” *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.
- [30] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [31] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu, “Asynchronous methods for deep reinforcement learning,” in *International Conference on Machine Learning*. PMLR, 2016, pp. 1928–1937.
- [32] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz, “Trust region policy optimization,” in *International Conference on Machine Learning*. PMLR, 2015, pp. 1889–1897.



- [33] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *International Conference on Machine Learning*. Pmlr, 2018, pp. 1861–1870.
- [34] S. Fujimoto, H. Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in *International Conference on Machine Learning*. PMLR, 2018, pp. 1587–1596.
- [35] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," *arXiv preprint arXiv:1509.02971*, 2015.
- [36] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.